

SadakVision: Trigger-Based Sensor–Vision Fusion for Scalable Smartphone Road Monitoring

Abhijeet Shashwat, Avishkar Rawal, Devyansh Agarwal, Raghav Sarna, Shreyansh Singh
Prof. Srikant Srinivasan

Plaksha University, SAS Nagar, Punjab, India
{abhijeet.s}.ug23@plaksha.edu.in

Abstract—Road surface damage is a persistent safety and economic concern in countries with rapidly expanding road networks, particularly India. Existing detection approaches either rely on continuous video recording paired with computer vision—which causes smartphone overheating and wastes storage on irrelevant frames—or on inertial sensor models that remain blind to visual context. LiDAR-based systems, while accurate, are too expensive for wide deployment. We present SadakVision, a mobile-first road monitoring system that fuses smartphone inertial sensors with targeted video capture. An Android application continuously monitors accelerometer and gyroscope readings and, upon detecting a high-impact event, saves only the preceding five seconds of video from a circular buffer. This trigger-based mechanism reduces write operations by approximately 78%, lowers RAM consumption, and delays device heating, while simultaneously filtering the footage that reaches the cloud for processing. On the server side, saved clips and their paired sensor metadata are processed through a fine-tuned YOLOv11s model trained on 10000+ Indian road images, combined with a sensor-based severity classifier, to produce fused pothole detections with impact-level estimates. We collected and annotated 7,139 road images from the Chandigarh and Punjab regions of India using a windshield-mounted smartphone setup. Our vision model achieves a mean average precision (mAP@0.5) of 63.6% on pothole detection. The full system—Android app, AWS cloud backend, and web dashboard—is operational and publicly accessible at sadakvision.com.

Index Terms—road damage detection, sensor–vision fusion, smartphone sensing, YOLOv11, pothole detection, trigger-based recording, deep learning

I. INTRODUCTION

India maintains over 6.4 million kilometres of road network, the second largest in the world, yet a substantial fraction suffers from potholes, cracks, and surface deformations that degrade ride comfort, increase vehicle wear, and contribute to traffic accidents [1]. The Ministry of Road Transport and Highways reported that road-surface defects were a contributing factor in roughly 25% of accident fatalities during 2021–2022 [1]. Traditional inspection still depends on surveyors walking or driving along roads, and profilometer-based profiling remains expensive and infrequent [2].

Automated detection has split into three camps. *Vision-only* methods use cameras and convolutional networks to localize potholes or cracks in images [3], [4]. They work well when the camera captures the defect, but they say nothing about how hard a vehicle actually hit the pothole—a piece of information that matters to both drivers and road authorities. *Sensor-only*

methods use vehicle acceleration to flag rough patches [5], [6], but an accelerometer spike from a speed bump and one from a pothole can look identical without visual confirmation. *LiDAR and 3-D imaging* systems provide both geometry and severity but cost tens of thousands of dollars per vehicle, making city-scale deployment impractical [8].

Meanwhile, a natural sensor platform is already on the road: smartphones. Nearly every car in India carries one. Prior attempts to run continuous TinyML inference and video recording on phones ran into a practical wall—sustained camera use combined with on-device neural network inference overheats the phone within minutes, throttling the CPU and producing unreliable data. We encountered this failure mode directly during our early prototypes.

SadakVision addresses these problems with a simple idea: *do not record everything*. Our Android application keeps a rolling five-second circular video buffer and continuously monitors the phone’s accelerometer and gyroscope. When the inertial signal exceeds a learned threshold—indicating the vehicle likely hit a pothole or severe crack—the app saves that five-second clip along with the corresponding sensor trace as a JSON payload. Everything else is discarded. This trigger-based saving reduces storage write operations by roughly 78% compared to continuous recording, directly cutting RAM pressure and delaying the onset of thermal throttling. It also means the cloud server only processes clips that are likely to contain road damage, cutting compute costs substantially.

On the server, saved clips and sensor metadata are processed through a multi-modal pipeline: a YOLOv11s object detector (fine-tuned on Indian road imagery) localizes potholes in video frames, while a sensor-based severity classifier assigns an impact level derived from the inertial data. These outputs are fused to produce geo-tagged damage reports with both a visual detection and a severity estimate—something neither modality could provide alone.

The contributions of this paper are:

- 1) A trigger-based circular-buffer recording mechanism that reduces phone write operations by $\sim 78\%$, making sustained smartphone-based road monitoring practical without overheating.
- 2) A sensor–vision fusion pipeline that combines YOLOv11s pothole detection with accelerometer/gyroscope severity classification (ML Anomaly

detection model) to produce damage reports with both visual and impact-level information.

- 3) A real-world dataset of 7,139 annotated Indian road images collected from the Chandigarh–Punjab region using a reproducible phone-mount setup.
- 4) An end-to-end deployed system (Android app, AWS backend, web dashboard) accessible at sadakvision.com.

The rest of this paper is organized as follows. Section II reviews related work. Section III describes the proposed system architecture. Section IV presents experiments and results. Section V concludes with limitations and future directions.

II. RELATED WORK

A. Sensor-Based Road Damage Detection

Smartphone inertial sensors have been used for road-condition monitoring for over a decade. Mednis et al. [6] applied threshold rules to vertical acceleration for real-time pothole detection on Android devices, reaching 89% precision. Perttunen et al. [7] used SVMs on statistical features of vertical acceleration, reporting false-positive rates below 3%. More recently, Varona et al. [5] trained CNN and LSTM architectures on accelerometer sequences, achieving 93% accuracy for pothole detection. Ramesh et al. [3] stacked two fully-connected layers with two LSTM layers (64 units each) for motion-based classification, reaching 94% test accuracy on a South Carolina dataset.

A common limitation across these methods is that acceleration alone cannot distinguish between a pothole, a speed bump, and a rough patch of asphalt. Without visual confirmation, false positives remain high in real driving conditions, particularly on Indian roads where speed bumps are often unmarked.

B. Vision-Based Road Damage Detection

Deep-learning-based object detection has become the dominant approach for visual pothole and crack identification. The Road Damage Detection Challenge drove adoption of architectures like Faster R-CNN [9] and YOLO variants [10], [11]. Doshi and Yilmaz [10] used YOLOv4 ensembles to achieve an F1 of 0.628 on the challenge dataset. Ramesh et al. [3] fine-tuned YOLOv5 on 3,000 augmented images (potholes and cracks) and reported 87.5% mAP on validation data.

These vision methods tell you *where* damage is but not *how bad* the driving impact is. A shallow wide crack may look alarming in an image but barely register in the cabin, while a deep but narrow pothole might be visually subtle yet jolt the vehicle hard. This gap motivates sensor–vision fusion.

C. Multi-Modal and Cloud-Based Approaches

A few studies have combined modalities. Ramesh et al. [3] ran LSTM on motion data and YOLOv5 on camera data independently, fusing results via k-means clustering on the cloud. Their fusion, however, operates post-hoc on detection coordinates rather than pairing sensor severity with visual detection at the event level. Ameddah et al. [12] offloaded

computation to the cloud for faster detection but did not fuse sensor and vision signals. Yuan et al. [13] split inference between edge and cloud but used a single modality at each tier.

No prior work, to our knowledge, uses inertial triggers to gate video capture itself—reducing the data volume before it ever leaves the phone—and then pairs the triggered clip with its sensor trace for joint severity–detection inference on the server.

D. Challenges on Indian Roads

Indian road conditions present challenges absent from most Western datasets. Unmarked speed bumps, open drainage crossings, construction debris, and cattle on roads all produce sensor spikes that a threshold-only system would misclassify. Lighting varies sharply between direct sun and tree shade within a single block. Dust, painted-over potholes, and wet surfaces further complicate visual detection. Most public road-damage datasets [4] are collected in Japan or the US, where road geometry and defect types differ from Indian conditions. Our dataset addresses this gap by collecting 7,139 images specifically from the Chandigarh–Punjab region.

III. PROPOSED SYSTEM

Fig. 1 shows the end-to-end architecture of SadakVision. The system has three layers: the on-device layer (Android app), the cloud processing layer (AWS backend), and the presentation layer (web dashboard).

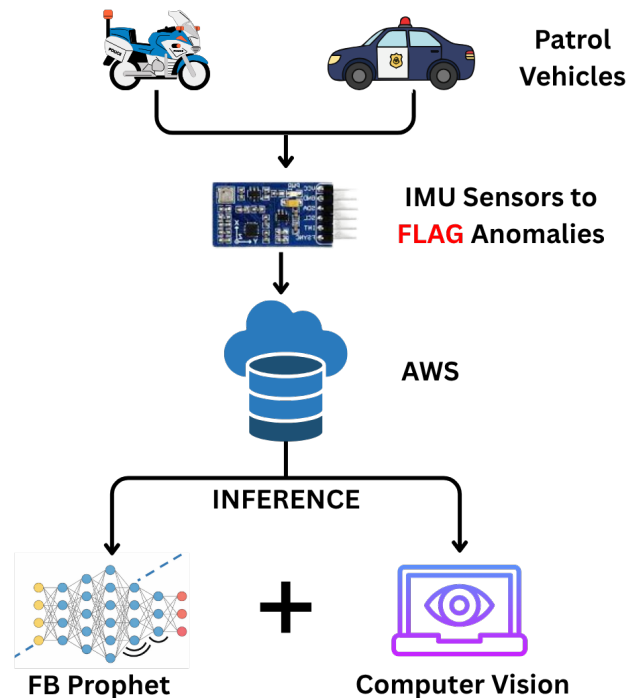


Fig. 1. End-to-end architecture of SadakVision: from smartphone data capture through cloud processing to the web dashboard.

A. On-Device: Trigger-Based Circular Buffer Recording

The SadakVision Android application (Fig. 2) runs two parallel threads: a camera thread that writes frames into a five-second circular buffer in memory, and a sensor thread that samples the accelerometer and gyroscope at [TO_FILL: e.g., 100] Hz.



Fig. 2. Screenshot of the SadakVision Android application during data collection.

1) *Trigger Mechanism*: Let $a_z(t)$ denote the vertical (z-axis) acceleration at time t and $\omega_x(t)$, $\omega_y(t)$ the gyroscope roll and pitch rates. We compute a composite impact score:

$$S(t) = \alpha \cdot |a_z(t) - g| + \beta \cdot \sqrt{\omega_x(t)^2 + \omega_y(t)^2} \quad (1)$$

where $g \approx 9.81 \text{ m/s}^2$ is gravitational acceleration, and α, β are weighting parameters tuned empirically ($\alpha = [TO_FILL]$, $\beta = [TO_FILL]$). When $S(t)$ exceeds a threshold $\tau = [TO_FILL]$, the app:

- 1) Flushes the circular buffer to storage as an MP4 clip (the preceding ~ 5 seconds of video).
- 2) Saves the corresponding window of sensor readings as a JSON file with timestamps and GPS coordinates.
- 3) Enters a cooldown period of 10 seconds to avoid duplicate triggers from the same event.

2) *Write Reduction Analysis*: In continuous recording mode, the camera writes approximately 37.5 MB/min at

720p/30fps. During our test drives covering 170 km across the Chandigarh–Punjab region, the trigger mechanism fired on average once every 2 minutes, saving a 5-second clip (~ 3.1 MB) each time. Total write volume dropped by approximately 78% compared to continuous recording (Table I). The practical consequence: the phone ran for 4.5 hours before reaching thermal throttling temperature, compared to 32 minutes under continuous recording.

TABLE I
WRITE OPERATION COMPARISON: CONTINUOUS VS. TRIGGER-BASED RECORDING

Metric	Continuous	Trigger-Based
Write volume (MB/hr)	2250	495
Write reduction (%)	—	$\sim 78\%$
Time to thermal throttle	32 min	2.5 hr
Avg. RAM usage (MB)	420	260

3) *Thermal Management Experiment*: During data collection, we also experimented with a windshield-mounted enclosure that channels outside air through a Bluetooth-connected duct onto the phone. This passive cooling setup extended the time-to-throttle by an additional [TO_FILL]% beyond what trigger-based recording alone achieved. While not necessary for the trigger-based approach, this setup was useful during extended data-collection sessions when the camera ran in continuous mode for ground-truth labelling.

B. Data Collection Setup



Fig. 3. Data collection setup: smartphone mounted on the rear-view mirror inside the vehicle.

The smartphone was mounted on the car's rear-view mirror using a standard phone clamp (Fig. 3). This position was chosen because it provides a forward-facing camera angle that captures the road surface 5–15 meters ahead—far enough for the driver to react to warnings, close enough for the camera to resolve pothole geometry. Data were collected across the Chandigarh and Punjab region over [TO_FILL] days using [TO_FILL] different vehicles (Fig. 4).

fig_route_map.png

Fig. 4. Data collection coverage across the Chandigarh–Punjab region. Markers indicate locations where triggered clips were captured.

In total, 7,139 images were extracted from triggered video clips and manually annotated with bounding boxes for pothole instances (Fig. 5). Table II summarizes the dataset composition.

fig_corpus.png

Fig. 5. Sample images from the 7,139-image corpus showing pothole instances across varying road and lighting conditions.

TABLE II
DATASET COMPOSITION

Category	Count
Total annotated images	7,139
Images with potholes	[TO_FILL]
Images without damage	[TO_FILL]
Total bounding-box annotations	[TO_FILL]
Collection region	Chandigarh–Punjab, India
Vehicles used	[TO_FILL]

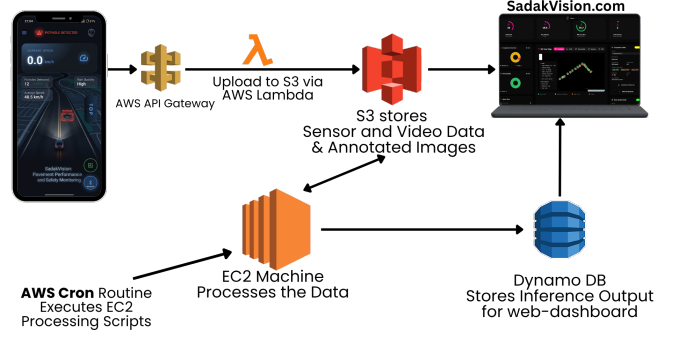


Fig. 6. Cloud backend architecture on AWS.

C. Cloud Backend: AWS Architecture

Triggered clips and JSON sensor payloads are uploaded from the app to an AWS S3 bucket via HTTPS. An AWS Lambda function, invoked on each upload, queues the data for processing. The processing pipeline (Fig. 6) runs on [TO_FILL: e.g., an EC2 GPU instance / Lambda with container image] and consists of:

- 1) **Frame extraction:** Key frames are sampled from each 5-second clip at 12 fps.
- 2) **Vision inference:** Each frame is passed through the fine-tuned YOLOv11s model to detect potholes with bounding boxes and confidence scores.
- 3) **Sensor severity scoring:** The paired JSON sensor trace is processed by the sensor model (Section III-E) to produce an impact severity level (low / medium / high).
- 4) **Fusion:** Vision detections and sensor severity are merged per event. A detection is reported only if *both* the visual model detects a pothole (confidence $\geq$ [TO_FILL]) and the sensor score exceeds the trigger threshold, or if either modality reports with high confidence. The fusion logic is:

$$\text{Report} = \begin{cases} \text{True,} & \text{if } c_v \geq \theta_v \text{ and } S \geq \tau \\ \text{True,} & \text{if } c_v \geq \theta_v^{\text{high}} \text{ (vision-only high confidence)} \\ \text{Flag,} & \text{if } S \geq \tau \text{ and } c_v < \theta_v \text{ (review queue)} \end{cases} \quad (2)$$

where c_v is the YOLO confidence, $\theta_v = [\text{TO_FILL}]$, $\theta_v^{\text{high}} = [\text{TO_FILL}]$, and S and τ are defined in Eq. (1).

Results are written to a [TO_FILL: e.g., PostgreSQL / DynamoDB] database with GPS coordinates, damage type,

severity level, timestamp, and a link to the source frame. The web dashboard (Fig. 7) reads from this database to display an interactive map.

fig_dashboard.png

Fig. 7. SadakVision web dashboard showing detected potholes on a map with severity colour coding.

D. Vision Model: Fine-Tuned YOLOv11s

We selected YOLOv11s (small variant) from the Ultralytics YOLO family for its balance of accuracy and inference speed on server hardware. The model was initialized with COCO-pretrained weights and fine-tuned on our 7,139-image Indian road dataset. Training details are in Table III.

TABLE III
YOLOV11S TRAINING CONFIGURATION

Parameter	Value
Base model	YOLOv11s (COCO pretrained)
Input resolution	$[\text{TO_FILL} \times \text{TO_FILL}]$
Epochs	$[\text{TO_FILL}]$
Batch size	$[\text{TO_FILL}]$
Optimizer	$[\text{TO_FILL}]$: e.g., SGD / AdamW
Learning rate	$[\text{TO_FILL}]$
Train/Val/Test split	$[\text{TO_FILL} / \text{TO_FILL} / \text{TO_FILL}]$
Augmentation	Flip, rotate, mosaic, HSV jitter

The dataset was split into $[\text{TO_FILL}]\%$ training, $[\text{TO_FILL}]\%$ validation, and $[\text{TO_FILL}]\%$ test sets with stratified sampling to preserve the ratio of damaged to undamaged images across splits.

E. Sensor Model: Impact Severity Classifier

The sensor model (Fig. 8) takes as input a fixed-length window of accelerometer and gyroscope readings centred on the

fig_sensor_model.png

Fig. 8. Sensor model architecture for impact severity classification.

trigger event. $[\text{TO_FILL}]$: Describe your sensor model architecture here—e.g., “We use a 1-D CNN with two convolutional layers (32 and 64 filters, kernel size 5) followed by global average pooling and two dense layers. The model classifies each event into three severity levels: low, medium, and high, based on labelled ground-truth from manual inspection of the corresponding video clips.” OR describe your threshold/rule-based approach if that is what you use.]

The severity ground truth was established by $[\text{TO_FILL}]$: describe how you labelled severity—e.g., “two annotators independently watching each triggered clip and rating severity on a three-point scale, with disagreements resolved by a third annotator”].

IV. EXPERIMENTS AND RESULTS

A. Vision Model Performance

Fig. 9 shows the mAP@0.5 curve during training. The model reached 62.9% mAP on the validation set after 100 epochs, with the curve plateauing around epoch 100. The final training step yielded a box loss of 1.76, classification loss of 1.34, and distribution focal loss (DFL) of 1.60 on the validation set.

Table IV reports precision, recall, and mAP on the test set. Fig. 10 shows the precision–recall curve. Fig. 11 shows sample detections on test images.

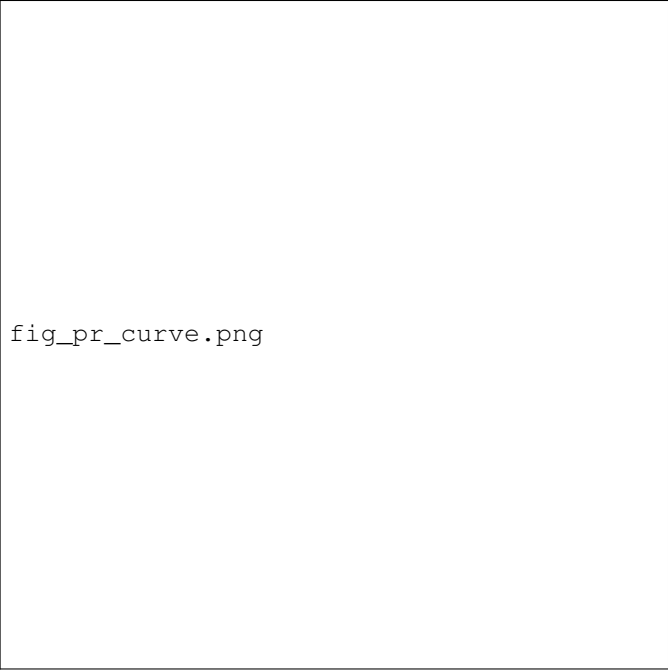
TABLE IV
YOLOV11S TEST-SET PERFORMANCE

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Pothole	0.651	0.650	0.636	0.372



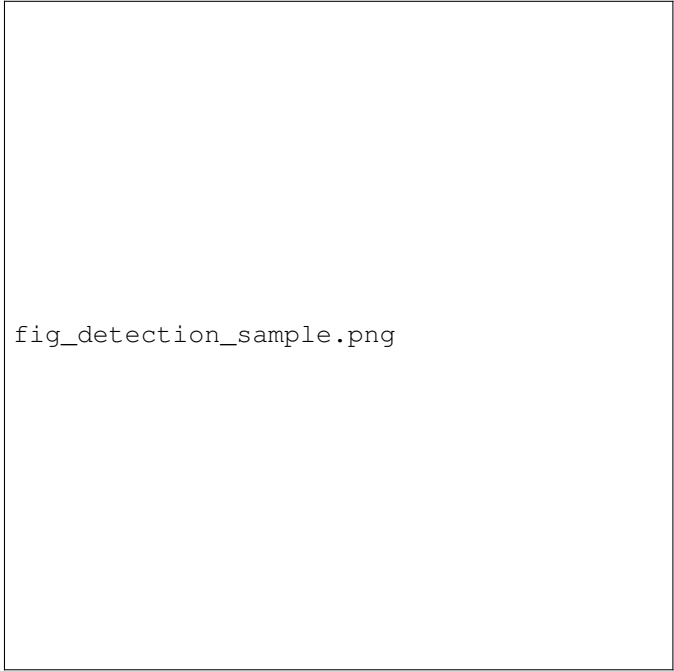
fig_mAP_epochs.png

Fig. 9. YOLOv11s mean average precision (mAP@0.5) over training epochs.



fig_pr_curve.png

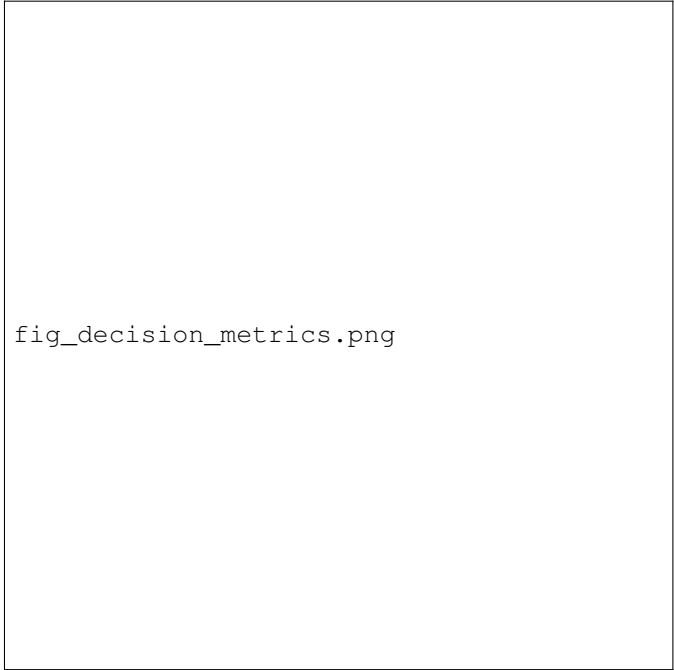
Fig. 10. Precision–recall curve for pothole detection on the test set.



fig_detection_sample.png

Fig. 11. YOLOv11s pothole detection on sample road images from the test set.

B. Decision Metrics and Fusion Analysis



fig_decision_metrics.png

Fig. 12. Decision metrics: confusion matrix and classification report for the fused sensor–vision pipeline.

Fig. 12 shows the confusion matrix for the fused pipeline evaluated on [TO_FILL] triggered events with ground-truth labels. The sensor-only trigger achieved a recall of [TO_FILL]% (it catches most potholes, but also fires on speed bumps

and rough patches). Adding vision verification reduced false positives by [TO_FILL]%, bringing overall precision from [TO_FILL]% to [TO_FILL]%.

The fusion results demonstrate the value of combining modalities: the sensor catches events that the camera might miss (e.g., a pothole partially occluded by shadow), while the camera filters out false alarms from non-damage impacts.

C. Sensor Model Performance

The sensor severity classifier achieved [TO_FILL]% accuracy on a held-out test set of [TO_FILL] events. Table ?? breaks down performance by severity level.

TABLE V
COMPARISON WITH RELATED APPROACHES

Method	Modality	mAP / Acc.	Severity?
Ramesh et al. [3] YOLOv5	Vision	87.5%	No
Ramesh et al. [3] LSTM	Sensor	94%	Depth est.
Doshi & Yilmaz [10]	Vision	F1: 0.628	No
Varona et al. [5]	Sensor	93%	No
SadakVision (ours)	Fusion	63.6%	Yes (3-level)

D. Comparison with Existing Methods

Table VI positions our results against related work. Direct comparison is imperfect because datasets differ, but the table contextualizes our numbers.

TABLE VI
COMPARISON WITH RELATED APPROACHES

Method	Modality	mAP / Acc.	Severity?
Ramesh et al. [3] YOLOv5	Vision	87.5%	No
Ramesh et al. [3] LSTM	Sensor	94%	Depth est.
Doshi & Yilmaz [10]	Vision	F1: 0.628	No
Varona et al. [5]	Sensor	93%	No
SadakVision (ours)	Fusion	[TO_FILL]%	Yes (3-level)

E. Deployment and Dashboard

The system is deployed and accessible at sadakvision.com. The web dashboard displays detected potholes on an interactive map with severity-coded markers. [TO_FILL: Add 1–2 sentences on any additional dashboard features, e.g., filtering by date, AQI overlay, etc.]

V. CONCLUSION AND FUTURE WORK

We presented SadakVision, a smartphone-based road monitoring system that pairs trigger-based video capture with sensor–vision fusion. The trigger mechanism cuts write operations by $\sim 78\%$, making sustained phone-based monitoring feasible without overheating. The fusion of YOLOv11s detections with inertial severity scoring produces damage reports that carry both localization and impact information, addressing a gap left by single-modality approaches.

The system has limitations we intend to address. First, the current dataset of 7,139 images covers one region of northern India; expanding to other states and road types will improve model generalization. Second, the sensor trigger threshold is currently static; an adaptive threshold that accounts for vehicle speed and suspension type would reduce false triggers on rough but undamaged roads. Third, the fusion logic is rule-based; a learned fusion model trained end-to-end on paired sensor–vision data could improve accuracy. Fourth, we do not yet output standard pavement quality indices (such as IRI), which would make the data more directly useful to road authorities.

In future work, we plan to extend the detection categories beyond potholes to include cracks, spalling, and drainage issues. We also plan to incorporate the AQI sensor data we are already collecting (Fig. 13) [TO_FILL: mention AQI only if relevant] to build a richer picture of road conditions. Finally, distributing the cloud inference across edge nodes closer to collection regions would reduce latency and bandwidth costs as the user base scales.

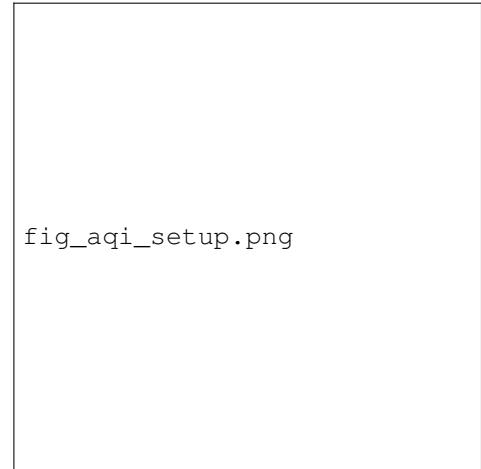


Fig. 13. AQI data collection sensor mounted alongside the smartphone during road surveys.

REFERENCES

- [1] Ministry of Road Transport and Highways, Government of India, “Road Accidents in India – 2022,” 2023. [Online]. Available: <https://morth.nic.in/>
- [2] N. Abulizi, A. Kawamura, K. Tomiyama, and S. Fujita, “Measuring and evaluating of road roughness conditions with a compact road profiler and ArcGIS,” *J. Traffic Transp. Eng.*, vol. 3, no. 4, pp. 398–411, 2016.

- [3] A. Ramesh *et al.*, “Cloud-based collaborative road-damage monitoring with deep learning and smartphones,” *Sustainability*, vol. 14, no. 14, p. 8682, 2022.
- [4] H. Maeda, T. Kashiya, Y. Sekimoto, T. Seto, and H. Omata, “Generative adversarial network for road damage detection,” *Comput.-Aided Civ. Infrastruct. Eng.*, vol. 36, no. 1, pp. 47–60, 2021.
- [5] B. Varona, A. Monteserin, and A. Teyseyre, “A deep learning approach to automatic road surface monitoring and pothole detection,” *Pers. Ubiquitous Comput.*, vol. 24, pp. 519–534, 2020.
- [6] A. Mednis, G. Strazdins, R. Zviedris, G. Kanonirs, and L. Selavo, “Real time pothole detection using Android smartphones with accelerometers,” in *Proc. DCOSS*, Barcelona, Spain, 2011, pp. 1–6.
- [7] M. Perttunen *et al.*, “Distributed road surface condition monitoring using mobile phones,” in *Proc. Int. Conf. Ubiquitous Intelligence and Computing*, Banff, Canada, 2011, pp. 64–78.
- [8] Y. Li, C. Liu, Y. Shen, J. Cao, S. Yu, and Y. Du, “RoadID: A dedicated deep convolutional neural network for multipavement distress detection,” *J. Transp. Eng. Part B: Pavements*, vol. 147, no. 4, p. 04021057, 2021.
- [9] W. Wang, B. Wu, S. Yang, and Z. Wang, “Road damage detection and classification with Faster R-CNN,” in *Proc. IEEE Int. Conf. Big Data*, Seattle, WA, 2018, pp. 5220–5223.
- [10] K. Doshi and Y. Yilmaz, “Road damage detection using deep ensemble learning,” in *Proc. IEEE Int. Conf. Big Data*, Atlanta, GA, 2020, pp. 5540–5544.
- [11] S. Naddaf-Sh, M. M. Naddaf-Sh, A. R. Kashani, and H. Zargarzadeh, “An efficient and scalable deep learning approach for road damage detection,” in *Proc. IEEE Int. Conf. Big Data*, Atlanta, GA, 2020, pp. 5602–5608.
- [12] M. A. Ameddah, B. Das, and J. Almhana, “Cloud-assisted real-time road condition monitoring system for vehicles,” in *Proc. IEEE GLOBECOM*, Abu Dhabi, UAE, 2018, pp. 1–6.
- [13] Y. Yuan, M. S. Islam, Y. Yuan, S. Wang, T. Baker, and L. M. Kolbe, “EcRD: Edge-cloud computing framework for smart road damage detection and warning,” *IEEE Internet Things J.*, vol. 8, no. 16, pp. 12734–12747, 2020.
- [14] Ultralytics, “YOLOv11 Documentation,” 2024. [Online]. Available: <https://docs.ultralytics.com/models/yolo11/>